

Segunda Lista de Exercícios - Programação Concorrente

****NOME**** Davi dos Santos Mattos ****DRE****: 119133049
Prof. Gabriel P. Silva - 2026.1

1) Qual a diferença entre o valor retornado pelas seguintes rotinas dentro e fora de uma região paralela? (0,5 ponto)

a) `omp_get_num_threads()`

R: Retorna o número de threads que estão atualmente ativas. Fora da região paralela retorna 1, pois apenas a thread principal está executando. Dentro da região paralela retorna a quantidade de threads ativas no grupo criador por `#pragma omp parallel`.

b) `omp_get_max_threads()`

R: Retorna o número máximo de threads que o OpenMP disponibilizará caso uma região paralela seja iniciada logo em seguida sem especificar um número exato.

Fora da região: Retorna o valor máximo padrão do sistema (geralmente igual ao número de núcleos do seu processador).

Dentro da região: Geralmente retorna o mesmo valor máximo configurado para o ambiente, pois é uma consulta de capacidade, não do estado atual.

c) `omp_get_num_procs()`

R: Retorna o número de processadores (núcleos lógicos) físicos disponíveis na máquina onde o código está rodando.

Fora e Dentro da região retorna exatamente o mesmo valor.

d) `omp_in_parallel()`

R: Retorna um valor booleano (verdadeiro ou falso) indicando se o código atual está sendo executado por múltiplas threads em paralelo. Fora da região retorna 0 (Falso). Dentro da região retorna 1 (Verdadeiro).

2) Quais as diferenças entre a declaração de variáveis `private`, `firstprivate` e `lastprivate`? Em quais diretivas são utilizadas? (0,5 ponto)

3) Quais as diferenças entre as diretivas `master` e `single`? (0,5 ponto)

4) Forneça um exemplo do uso da cláusula `reduction` no OpenMP. Descreva quatro tipos de operações de redução que podem ser utilizadas com esta cláusula. (0,5 ponto)

5) No código abaixo, faça um diagrama do escalonamento das iterações do laço entre 4 threads. (0,5 ponto)

C

```
#pragma omp for schedule(guided, 5) // [cite: 13]
for (j = 0; j < 50; j++) // [cite: 14, 15]
    a[j] = b[j] + c[j]; // [cite: 16]
```

6) Reescreva o código seguinte, colocando as diretivas `barrier` explicitamente, de modo a maximizar o desempenho, mas sem alterar o resultado da execução. (0,5 ponto)

C

```
#pragma omp parallel
{ // [cite: 19]
    #pragma omp for nowait // [cite: 20]
    for (j = 0; j < n; j++) // [cite: 21]
        a[j] = b[j] + c[j]; // [cite: 22]

    #pragma omp for nowait // [cite: 23]
    for (j = 0; j < n; j++) // [cite: 24]
        d[j] = e[j] * f; // [cite: 25]

    #pragma omp for nowait // [cite: 26]
    for (j = 0; j < n; j++) { // [cite: 27]
        z[j] = (a[j] + a[j+1]) * 8.5; // [cite: 29]
    } // [cite: 28]
}
```

7) Paralelizar o exemplo a seguir sem o uso da diretiva `for` ou `parallel for`. (0,5 ponto)

C

```
for (i = 0; i < n; i++) // [cite: 31]
    z[i] = a * x[i] + y; // [cite: 32]
```

8) Explique o porquê cada um dos laços a seguir pode ou não pode ser paralelizado com a diretiva `parallel for`. (0,5 ponto)

a)

C

```
for (i = 0; i < N; i++) // [cite: 35]
    if (x[i] > maxval) break; // [cite: 36]
```

b)

C

```
for (i = 0; i < N; i++) // [cite: 37]
    for (j = 0; j < i; j++) // [cite: 38]
        a[j][i] = a[j+1][i]; // [cite: 39]
```

c)

C

```
for (k = 0; k < N; k++) // [cite: 40]
    x[k] = q + y[k] * (r * z[k+10] + t * z[k+10]); // [cite: 41]
```

d)

C

```
for (i = 1; i < N; i++) // [cite: 42]
    x[i] = z[i] * (y[i] - x[i-1]); // [cite: 43]
```

e)

C

```
for (i = 0; i < N; i++) { // [cite: 44]
    if (fabs(a[i]) > machine_max || fabs(a[i]) < machine_min) { // [cite:
45, 47]
        printf("i=%d \n", i); // [cite: 48]
        break; // [cite: 49]
    } // [cite: 50]
    a[i] = a[i] * a[i]; // [cite: 46]
} // [cite: 51]
```

f)

C

```
for (i = 1; i < N; i++) // [cite: 52]
    for (k = 0; k < i; k++) // [cite: 54]
        w[i] += b[k][i] * w[(i-k)-1]; // [cite: 53]
```

g)

C

```
for (k = 0; k < N; k++) // [cite: 55]
    x[k] = u[k] + r * (z[k] + r * y[k]) + // [cite: 56]
```

```
t * (u[k+3] + r * (u[k+2] + r * u[k+1])) + // [cite: 57]
t * (u[k+6] + r * (u[k+5] + r * u[k+4]))); // [cite: 58]
```

9) Considere o seguinte laço: (0,5 ponto)

C

```
x = 1; // [cite: 60]
#pragma omp parallel for firstprivate(x) // [cite: 61]
for(i = 0; i < N; i++) { // [cite: 62]
    y[i] = x + i; // [cite: 64]
    x = i; // [cite: 65]
} // [cite: 63]
```

a) Porque este laço está incorreto? `y[i]` recebe o mesmo resultado independente do número de threads executando o laço? b) Qual o valor da variável `i` ao final do laço? Qual é o valor da variável `x` ao final do laço? c) Qual seria o valor de `x` ao final do laço se seu escopo fosse `shared`? d) Este laço pode ser paralelizado corretamente (isto é, preservando a semântica sequencial) apenas com o uso de diretivas OpenMP?

10) Quais os mecanismos disponíveis no OpenMP para lidar com as condições de corrida? (0,5 ponto)

11) Descreva como a diretiva `ordered` é utilizada no OpenMP. Apresente um trecho de código como exemplo. (0,5 ponto)

12) Como a cláusula `collapse` funciona no OpenMP e em que situações deve-se utilizá-la? (0,5 ponto)

13) Explique a diferença fundamental entre tarefas explícitas e tarefas implícitas no OpenMP. Em que contextos cada tipo de tarefa é gerado? (0,5 ponto)

14) Defina o que é um ponto de escalonamento de tarefas. Liste pelo menos três ações distintas que uma thread pode tomar ao encontrar um ponto de escalonamento durante a execução de uma tarefa. (0,5 ponto)

15) O que são os pontos de sincronização de tarefas, e como a diretiva `#pragma omp taskwait` é especificamente utilizada neste contexto? (0,5 ponto)

16) No trecho de código a seguir, identifique adequadamente os pontos de escalonamento e de sincronização de código. (1,0 ponto)

C

```
#include <stdio.h> // [cite: 83]
#include <omp.h> // [cite: 84]

int main() { // [cite: 84]
```

```

#pragma omp parallel num_threads(4) // [cite: 85]
{ // [cite: 86]
    #pragma omp single // [cite: 87]
    { // [cite: 88]
        #pragma omp task // [cite: 89]
        printf("Executando Tarefa A \n"); // [cite: 90]

        #pragma omp taskyield // [cite: 91]
        printf("Após taskyield \n"); // [cite: 92]

        #pragma omp task // [cite: 93]
        printf("Executando Tarefa B \n"); // [cite: 94]

        #pragma omp taskwait // [cite: 95]
        printf("Após taskwait \n"); // [cite: 96]

        int data = 0; // [cite: 97]

        #pragma omp task depend(out: data) // [cite: 98]
        data = 42; // [cite: 98]

        #pragma omp task depend(in: data) // [cite: 99]
        printf("Data = %d\n", data); // [cite: 100]

        #pragma omp taskgroup // [cite: 101]
        { // [cite: 102]
            #pragma omp task // [cite: 103]
            printf("Tarefa E no taskgroup \n"); // [cite: 104]

            #pragma omp task // [cite: 105]
            printf("Tarefa F no taskgroup \n"); // [cite: 106]
        } // Fim do taskgroup [cite: 107, 108]
    } // Fim do single [cite: 109, 110]
} // Fim do parallel [cite: 111, 112]
return 0; // [cite: 113]
} // Fim do programa [cite: 114]

```

17) Descreva a função das cláusulas `if` e `final` e como elas se diferenciam na geração e execução das tarefas. (0,5 ponto)

18) Considere o seguinte trecho de código. Aplique a cláusula `depend` com os argumentos corretos para garantir a execução correta das tarefas. Em seguida, desenhe o grafo de dependências. (0,5 ponto)

C

```

int main() { // [cite: 119]
    int A = 0, B = 0, C = 0; // [cite: 120]

```

```

#pragma omp parallel // [cite: 121]
{ // [cite: 122]
    #pragma omp single // [cite: 123]
    { // [cite: 124]
        #pragma omp task // [cite: 125]
        // Tarefa A [cite: 126]
        A = 1; // [cite: 125]

        #pragma omp task // [cite: 127]
        // Tarefa B [cite: 128]
        B = A + 1; // [cite: 129]

        #pragma omp task // [cite: 130]
        // Tarefa C [cite: 131]
        C = B + C; // [cite: 132]

        #pragma omp task // [cite: 133]
        // Tarefa D [cite: 134]
        printf("Resultado final %d %d %d \n", A, B, C); // [cite: 135]
    } // Fim do single [cite: 137]
} // Fim do parallel [cite: 136, 138]

return 0; // [cite: 140]
} // Fim do programa principal [cite: 139, 141]

```

19) Desenhe o grafo de dependências para o trecho de código a seguir: (1,0 ponto)

C

```

int main() { // [cite: 143]
    int x = 0, y = 8, z = 0; // [cite: 144]

    #pragma omp parallel // [cite: 145]
    { // [cite: 146]
        #pragma omp single // [cite: 148]
        { // [cite: 147]
            // Task 1: [cite: 149]
            #pragma omp task depend(out: x) shared(x) // [cite: 150]
            x = 5; // [cite: 151]

            // Task 2: [cite: 152]
            #pragma omp task depend(in: x, y, z) shared(x, y, z) // [cite:
153]

            int resultado = x + y + z; // [cite: 154]

            // Task 3: [cite: 155]
            #pragma omp task depend(in: x) depend(out: y) shared(x, y) //
[cite: 156]
            y = x * 2; // [cite: 159]

```

```

        // Task 4: [cite: 160]
        #pragma omp task depend(in: x) depend(out: z) shared(x, z) //
[cite: 161]
        z = x + 10; // [cite: 161]

        // Task 5: [cite: 162]
        #pragma omp task depend(inout: y) shared(y) // [cite: 163]
        y = y + 3; // [cite: 164]

        // Task 6: [cite: 165]
        #pragma omp task depend(inout: z) shared(z) // [cite: 166]
        z = z - 2; // [cite: 167]
    } // [cite: 157]
} // [cite: 158]

printf("Valores finais: x=%d, y=%d, z=%d\n", x, y, z); // [cite: 169]
return 0; // [cite: 169]
}

```

20) Explique a diretiva `#pragma omp taskloop`. Mencione pelo menos duas cláusulas específicas que podem ser usadas com `taskloop` e descreva suas funções. (0,5 ponto)

Glossário

- **OpenMP (Open Multi-Processing):** Uma interface de programação (API) para escrever aplicações multithread em memória compartilhada em linguagens como C, C++ e Fortran.
- **Diretiva/Pragma:** Instruções especiais que o compilador utiliza (como `#pragma omp`) para alterar o comportamento de execução (ex: paralelizar um bloco).
- **Condição de corrida (Data Race):** Ocorre quando duas ou mais threads tentam acessar e modificar o mesmo dado simultaneamente, gerando resultados imprevisíveis.
- **Barreira (Barrier):** Ponto de sincronização onde todas as threads de uma equipe devem parar e aguardar até que todas as outras threads também alcancem este ponto.

Pontos de atenção

- **Dependências de dados em laços (Exercício 8):** Nem todo laço `for` pode ser magicamente paralelizado. Se a iteração atual (ex:) depende de um valor calculado na iteração anterior (ex:), o OpenMP não conseguirá paralelizar sem gerar uma condição de corrida ou quebrar a lógica matemática.
- **Fuga de Escopo:** Variáveis criadas fora do bloco paralelo por padrão são `shared` (compartilhadas). Modificá-las concorrentemente sem diretivas adequadas como `atomic` ou `reduction` fatalmente corromperá seu cálculo.
- **Quebra de fluxo estruturado (break/return):** O OpenMP exige que o bloco paralelo tenha um único ponto de entrada e de saída no fluxo. Utilizar comandos como `break`

(como visto no Exercício 8e) ou `return` no meio de um laço paralelizado resulta em falha de compilação.

Resumo do método

Para abordar as resoluções numéricas e conceituais desta lista de forma sólida, siga este checklist durante seus estudos:

1. **Identifique a Região Paralela:** Observe onde começa o `#pragma omp parallel` e onde ele termina.
2. **Mapeie o Escopo das Variáveis:** Liste quais variáveis são `shared`, `private`, `firstprivate` ou `lastprivate`.
3. **Desenhe Grafos de Dependência (para diretivas `task`):** Se existirem cláusulas `depend(in)`, `depend(out)` ou `depend(inout)`, desenhe nós (tarefas) e setas apontando quem precisa terminar para quem começar.
4. **Inspecione Dependências Cíclicas:** Em laços `for`, escreva manualmente as três primeiras iterações (ex: iteração) para visualizar se há leitura de índices recém-modificados ou não.

Esta é toda a transcrição, formatada e corrigida de anomalias visuais de extração. Existe algum exercício específico desta lista cuja teoria e derivação passo a passo você gostaria de explorar agora?